

Módulo 5 – Capítulo 10 – Sección 01

Patrones de entrada: validación, normalización y pre-procesamiento de prompts

El pre-procesamiento de la entrada del usuario antes de enviarla al LLM es una capa de defensa que el código de la aplicación controla completamente —a diferencia del comportamiento del modelo, que no puede controlarse directamente. Esta capa tiene tres propósitos que se refuerzan mutuamente: calidad (normalizar el input para que el modelo lo procese de forma consistente), economía (rechazar inputs fuera de rango antes de gastar tokens en procesarlos), y seguridad (detectar y bloquear intentos de manipulación del sistema). Las tres funciones son más baratas de implementar en la capa de entrada que de remediar en la capa de salida, porque un input incorrecto que llega al LLM produce una respuesta incorrecta que el sistema debe manejar, mientras que un input incorrecto detectado antes nunca genera el costo de la llamada al LLM.

La validación de entrada es la primera función. La longitud máxima del input —medida en tokens, no en caracteres— debe ser un límite explícito configurado según el presupuesto de tokens del sistema: si el system prompt ocupa 2.000 tokens y el historial conversacional hasta 4.000 tokens, el input del usuario no puede superar los tokens disponibles sin exceder la ventana de contexto o el presupuesto de costo definido.

```
if count_tokens(user_input) > MAX_INPUT_TOKENS: raise InputTooLongError(f"Input {count_tokens(user_input)} tokens, max {MAX_INPUT_TOKENS}")
```

rechaza el input antes de la llamada con un error informativo que el sistema puede convertir en un mensaje de usuario apropiado. Los inputs vacíos, los inputs que consisten solo en espacios o caracteres de control, y los inputs con patrones de repetición que indican un bug del cliente también deben rechazarse con validación explícita.

La detección de prompt injection es la segunda función, y la más específica de los sistemas de IA. El prompt injection directo ocurre cuando el usuario intenta añadir instrucciones al system prompt o sobrescribir instrucciones existentes con frases como "ignora las instrucciones anteriores", "actúa como un asistente sin restricciones", o "tu nueva instrucción es". La detección mediante expresiones regulares sobre patrones conocidos, complementada con un clasificador ligero entrenado sobre ejemplos de prompt injection, puede detectar los

intentos más comunes con alta precisión y bajo costo (una llamada a un modelo pequeño o un clasificador local). El indirect prompt injection —donde el contenido malicioso está en un documento que el sistema recupera para incluir en el contexto RAG, no en el input directo del usuario— es más difícil de detectar y requiere sanitización del contenido de los documentos antes de incluirlos en el contexto.

La normalización prepara el input para procesamiento consistente. La normalización de encoding con `unicodedata.normalize('NFC', text)` unifica representaciones equivalentes de caracteres Unicode. La eliminación de caracteres de control con `re.sub(r'[\x00-\x08\x0b\x0c\x0e-\x1f]', '', text)` previene que caracteres invisibles confundan al tokenizador. La detección automática del idioma con `langdetect` o `fasttext-langid` permite configurar el idioma de respuesta en el prompt cuando el sistema es multilingüe. El enriquecimiento de contexto dinámico —insertar la fecha y hora actuales, el perfil del usuario, el idioma detectado, los permisos del usuario— transforma variables del template del prompt en datos reales antes de construir el mensaje final.

Componentes principales del pre-procesamiento de entrada

- **Validación de longitud en tokens:** `count_tokens(user_input) > MAX_INPUT_TOKENS` rechaza el input antes de la llamada; usar el mismo tokenizador del modelo (tiktoken para OpenAI, `client.messages.count_tokens()` para Anthropic) para precisión exacta.
- **Detección de prompt injection:** lista de patrones regex para variantes conocidas de "ignora las instrucciones anteriores" combinada con un clasificador ligero; los inputs detectados se loglean en el stream de seguridad, se rechazan con un mensaje de error apropiado, y opcionalmente se escalan para revisión.
- **Sanitización de encoding:** `text.encode('utf-8', errors='replace').decode('utf-8')` elimina caracteres inválidos; `unicodedata.normalize('NFC', text)` unifica representaciones equivalentes; `re.sub(r'[\x00-\x08\x0b\x0c\x0e-\x1f]', '', text)` remueve caracteres de control.
- **Enriquecimiento de contexto dinámico:** rellenar `{current_date}`, `{user_name}`, `{user_plan}`, `{detected_language}`, `{user_permissions}` en el template del prompt con datos del contexto de la sesión antes de construir el mensaje; hacer este paso antes de contar los tokens del prompt final.
- **Clasificación de intent de primer paso:** un modelo ligero (Haiku con un prompt de clasificación, o un clasificador KNN sobre embeddings) determina si la query es in-scope antes de procesar con el modelo principal; el costo de la clasificación de primer paso

(\$0.00003 con Haiku) es insignificante comparado con el costo evitado de procesar queries out-of-scope con el modelo completo.

Nota del Arquitecto: *La seguridad en sistemas de IA no es un dominio separado de la ingeniería del sistema: es una propiedad que debe diseñarse en la capa de entrada, en el sistema prompt, en la capa de salida, y en el sistema de logging, de forma integrada. El OWASP Top 10 for LLMs es el marco de referencia estándar para identificar los vectores de ataque más relevantes: prompt injection, insecure output handling, training data poisoning, model denial of service, sensitive information disclosure, y otros riesgos específicos de sistemas LLM. Integrar este marco en el proceso de revisión de seguridad del equipo desde el primer sprint es más efectivo que añadir controles de seguridad retroactivamente.*

El pre-procesamiento de entrada es la primera línea de defensa de la calidad y la seguridad del sistema; un input malformado, malicioso o fuera de alcance que llega al LLM es más caro de manejar —tokens desperdiciados, respuesta incorrecta, posible daño reputacional— que rechazarlo o corregirlo en esta etapa.

Principio rector: El pre-procesamiento de entrada es la primera línea de defensa de la calidad y la seguridad del sistema; validar longitud, detectar prompt injection, normalizar encoding y enriquecer con contexto dinámico en esta capa es siempre más económico que manejar las consecuencias de un input incorrecto que llegó al LLM.

Módulo 5 – Capítulo 10 – Sección 02

Patrones de salida: parseo estructurado, reintentos y fallbacks

El LLM produce texto; el sistema necesita datos. Esta brecha entre la salida natural del modelo y los tipos de datos que el resto de la aplicación consume es una de las fuentes de fragilidad más comunes en los sistemas de IA en producción: el código que parsea la respuesta del modelo asume un formato, el modelo produce un formato ligeramente diferente, y el sistema lanza una excepción que ningún test anticipó. El patrón de salida estructurada es el conjunto de técnicas que cierran esta brecha de forma sistemática: especificar el formato esperado en el prompt, usar las capacidades de structured output del proveedor cuando están disponibles, parsear y validar con Pydantic, y definir explícitamente qué hace el sistema cuando el parseo falla.

Las APIs modernas de LLM ofrecen tres mecanismos de structured output con garantías diferentes. El structured output nativo —disponible en OpenAI con `response_format=`
`{"type": "json_schema", "json_schema": {...}}` y en Google con `response_mime_type="application/json"` más `response_schema` — garantiza que el modelo produce JSON que conforma al schema especificado, usando constrained decoding en la capa de generación del modelo: el modelo elige solo tokens que son compatibles con el schema en cada paso. Esta garantía elimina la posibilidad de JSON malformado pero no garantiza que los valores del JSON sean semánticamente correctos. El JSON mode — `response_format=`
`{"type": "json_object"}` en OpenAI sin schema— garantiza JSON válido pero no el schema específico. El prompting explícito de JSON —instrucciones en el system prompt más ejemplos few-shot de JSON— no ofrece garantías formales pero funciona con cualquier proveedor.

Pydantic es la capa de validación semántica que complementa las garantías de formato. `model`
`= ResponseModel.model_validate_json(response_text)` parsea el JSON y valida simultáneamente que todos los campos requeridos existen, que los tipos son correctos, que los valores de enum están en la lista permitida, y que las constraints de valor (`gt=0`, `max_length=500`) se cumplen. Cuando la validación falla con `ValidationError`, el sistema

sabe exactamente qué campo tiene qué problema, con un mensaje de error estructurado que puede usarse para construir un prompt de corrección.

El retry con corrección es la pieza que convierte un error de parseo en una segunda oportunidad de éxito. El `OutputFixingParser` de LangChain implementa este patrón: cuando la respuesta falla la validación de Pydantic, construye un segundo prompt que incluye la respuesta original, el error de Pydantic, y la instrucción de corregir el error manteniendo el contenido. Esta segunda llamada tiene alta tasa de éxito para errores de formato (campo faltante, valor de tipo incorrecto) pero baja para errores semánticos (el modelo no tiene la información necesaria para responder con el schema requerido). Limitar el retry a dos intentos máximo previene el gasto de llamadas adicionales en casos donde el modelo estructuralmente no puede cumplir el schema.

La jerarquía de fallback define el comportamiento del sistema cuando todos los intentos fallan. La jerarquía correcta varía por dominio, pero un ejemplo común es: (1) respuesta parcial con los campos disponibles y `null` en los campos faltantes —si el sistema puede funcionar con información parcial—, (2) respuesta default del sistema como "No puedo responder a esta consulta en el formato requerido", o (3) escalamiento a revisión humana para respuestas que el sistema no puede procesar automáticamente.

Componentes técnicos del parseo de salida estructurada

- **Structured output nativo de OpenAI:**

`client.beta.chat.completions.parse(model=..., messages=..., response_format=ResponsePydanticModel)` devuelve el objeto Pydantic directamente; usar `response.choices[0].message.parsed` para acceder al objeto sin parseo manual; la `guaranteed JSON conformance` elimina errores de JSON malformado.

- **Validación con Pydantic:** definir el schema como `class Response(BaseModel):`

`field: str = Field(description="...")` con validators explícitos; `model_validate_json(raw_text)` parsea y valida en un paso; capturar `ValidationError` para construir el error detallado que informa el retry prompt.

- **Retry con prompt de corrección:** dos intentos máximo; el segundo prompt incluye: la respuesta original entre `<original_response>` tags, el error de Pydantic en texto plano, y la instrucción `"Fix the above response to conform to the required JSON schema. Return ONLY valid JSON."`.

- **OutputFixingParser en LangChain:**

`OutputFixingParser.from_llm(parser=PydanticOutputParser(pydantic_object=Model),`

`llm=model)` encapsula el retry automáticamente; útil en cadenas LCEL para mantener el flujo limpio.

- **Jerarquía de fallback:** (1) parse completo exitoso → respuesta normal; (2) parse parcial con `ValidationError` → retry con corrección; (3) segundo intento fallido → respuesta parcial o default, log del error completo para revisión; (4) error de API en ambos intentos → error de sistema con dead letter queue.

Nota del Arquitecto: Los errores de parseo de salida en producción raramente son aleatorios: ocurren sistemáticamente para ciertos tipos de queries o ciertos patrones de respuesta del modelo. El logging estructurado de todos los errores de parseo — incluyendo el texto de la respuesta que falló, el error de Pydantic, y el resultado del retry— permite identificar estos patrones sistemáticos y corregirlos en el prompt, en el schema, o en la lógica de validación. Un sistema que nunca loggea las respuestas que no puede parsear está perdiendo información crítica sobre sus propios puntos de fallo.

El parseo robusto de salida no es un detalle de implementación: es una propiedad de calidad del sistema que determina cuántos requests producen datos utilizables por el resto de la aplicación; un sistema que falla silenciosamente en el parseo —devolviendo un objeto vacío o un default sin loggear el error— produce bugs de datos difíciles de trazar cuya causa original fue una respuesta malformada del LLM que nadie detectó.

Principio de diseño: La robustez del parseo de salida es proporcional a la especificidad del schema, la calidad del prompt de structured output, y la integridad del sistema de fallback; los tres componentes deben diseñarse juntos desde el inicio porque un schema mal definido produce errores que el mejor retry no puede corregir.

Módulo 5 – Capítulo 10 – Sección 03

El patrón Gateway: punto de entrada único para llamadas al LLM

En un sistema de IA con múltiples features, múltiples modelos y múltiples proveedores, el código que llama directamente al SDK del proveedor desde cada punto del sistema crea un problema de mantenimiento que escala con la complejidad: si el equipo decide migrar de una librería a otra, cambiar el modelo por defecto, agregar logging centralizado, o implementar retry logic, debe hacer el cambio en decenas de puntos de llamada dispersos por el codebase. El patrón Gateway resuelve este problema estableciendo una clase o servicio único como mediador entre la aplicación y los proveedores de LLM: todo el código de la aplicación habla con el Gateway, y el Gateway habla con los proveedores. El resultado es que los cambios de proveedor, modelo, logging, rate limiting y circuit breaking se implementan una sola vez en el Gateway y se aplican automáticamente a todas las llamadas del sistema.

La `LLMGateway` class encapsula las responsabilidades que serían código duplicado si estuvieran dispersas: la selección del cliente correcto por proveedor (Anthropic, OpenAI, Google), el logging estructurado de cada llamada (tokens usados, costo calculado, latencia, feature que originó la llamada), el retry con backoff exponencial para errores transitorios, el circuit breaker para aislar proveedores con fallas, y el seguimiento del gasto acumulado contra el límite mensual configurado. Cada llamada al Gateway registra estos datos automáticamente, sin que el código de la aplicación deba implementarlos.

La sección de configuración del Gateway es crítica: la selección del modelo, el proveedor, y los límites de gasto son parámetros de configuración que deben poder cambiarse sin modificar el código de la aplicación. Una clase `LLMConfig` con los campos `provider`, `model`, `max_tokens`, `temperature`, y `monthly_budget_usd` —cargada desde un archivo YAML versionado en Git— permite cambiar el modelo por defecto de `claude-3-5-sonnet-20241022` a `claude-3-5-haiku-20241022` modificando una línea en el YAML, sin tocar el código del Gateway. Este nivel de configurabilidad es especialmente valioso durante la optimización de costos: permite comparar el comportamiento del sistema con diferentes modelos sin cambios de código.

LiteLLM Proxy lleva el patrón al extremo en arquitecturas de microservicios: en lugar de una clase gateway en la aplicación, es un servicio independiente que expone la API de OpenAI como interfaz unificada y traduce internamente a cualquier proveedor configurado. Cualquier cliente que entiende la API de OpenAI puede hablar con LiteLLM Proxy y obtener respuestas de Anthropic, Google, o cualquier modelo de código abierto via Ollama, sin ningún cambio en el código cliente. La ventaja para equipos grandes es que el Proxy puede desplegarse como un servicio central que el equipo de plataforma mantiene, mientras los equipos de producto consumen la API unificada sin gestionar credenciales ni lógica de retry.

Componentes técnicos del patrón Gateway

- **LLMGateway class:** constructor que inicializa los clientes por proveedor y carga la configuración; método `complete(prompt, config, metadata)` como punto único de entrada; routing interno por `config.provider` al cliente correspondiente; el `metadata` dict (feature, request_id, user_id) se inyecta en cada log estructurado.
- **Logging automático en el Gateway:** cada llamada logea `model`, `provider`, `input_tokens`, `output_tokens`, `cost_usd`, `latency_ms`, `cache_hit`, `feature`, `request_id` en JSON; el Gateway es el único lugar donde este log se emite, eliminando la duplicación.
- **Rate limiting en el Gateway:** `asyncio.Semaphore(MAX_CONCURRENT_CALLS)` limita la concurrencia por proveedor; `asyncio.sleep` con backoff exponencial cuando el proveedor devuelve 429; el Gateway absorbe la complejidad del rate limiting y expone una interfaz simple al código de la aplicación.
- **Límite de gasto mensual:** contador `self._monthly_spend` acumulado en cada llamada; `if self._monthly_spend + estimated_cost > self.config.monthly_budget_usd: raise BudgetExceededError()` antes de la llamada; reiniciar el contador al inicio de cada mes mediante un job scheduled; alertar cuando el gasto supera el 80% del límite.
- **LiteLLM Proxy como alternativa:** servicio independiente con UI web de configuración, soporte para más de cien modelos, logging centralizado, rate limiting por equipo o feature; ideal para organizaciones con múltiples equipos consumiendo LLMs y necesidad de control centralizado de costos y acceso.

Nota del Arquitecto: El patrón Gateway resuelve el problema de la dispersión de código de infraestructura, pero introduce un punto único de acoplamiento que puede convertirse en un cuello de botella si no se diseña con las consideraciones de

rendimiento correctas. El Gateway debe ser stateless (el estado de la sesión o el historial conversacional no pertenecen al Gateway), debe ser async-first para no bloquear el event loop, y debe tener tests de integración que verifican el comportamiento correcto cuando el proveedor devuelve errores, rate limits, y timeouts. Un Gateway sin tests de fallo es más peligroso que no tener Gateway: concentra el riesgo en un solo punto.

El Gateway es el lugar correcto para implementar cross-cutting concerns del sistema de LLM: logging, retry, rate limiting, cost tracking, y circuit breaking. Centralizar estas responsabilidades en el Gateway mantiene el código de la aplicación limpio y enfocado en la lógica de negocio, mientras el Gateway garantiza que la infraestructura de resiliencia y observabilidad se aplica consistentemente a todas las llamadas.

Principio de arquitectura: El patrón Gateway convierte los cross-cutting concerns del sistema LLM —logging, retry, rate limiting, cost tracking, circuit breaking— en responsabilidades centralizadas que se implementan una vez y se aplican a todas las llamadas; la alternativa de implementar estos concerns en cada punto de llamada al LLM es código duplicado que diverge con el tiempo.

Módulo 5 – Capítulo 10 – Sección 04

Circuit Breaker: resiliencia ante fallos del proveedor

Un sistema de IA que llama a un proveedor externo tiene una dependencia de disponibilidad que no puede controlar. Anthropic, OpenAI y Google tienen SLAs del 99.9% o del 99.5%, lo que implica entre 43 minutos y 4 horas de indisponibilidad potencial al año. Pero más allá de las interrupciones totales, los problemas parciales —alta latencia, errores 503 esporádicos, rate limiting severo durante picos de uso del proveedor— son más comunes y más dañinos para la experiencia del usuario porque producen respuestas lentas y errores intermitentes que son difíciles de diagnosticar. El patrón Circuit Breaker es el mecanismo de resiliencia que detecta cuándo un proveedor está degradado y para de enviarlo requests antes de que los usuarios experimenten timeouts repetidos, permitiendo que el sistema falle rápido con un mensaje de error apropiado en lugar de fallo lento con timeouts acumulados.

El Circuit Breaker tiene tres estados con transiciones automáticas basadas en métricas. En estado **Closed** (circuito cerrado, funcionamiento normal), todas las llamadas pasan al proveedor. Si el número de fallos consecutivos supera el umbral configurado —por ejemplo, cinco errores 500 o cinco timeouts en una ventana de treinta segundos—, el breaker transiciona a **Open** (circuito abierto): todas las llamadas se rechazan inmediatamente con `CircuitOpenError` sin intentar llamar al proveedor, fallando rápido con un mensaje de error apropiado. Después de un timeout de recuperación configurado —por ejemplo, treinta segundos o un minuto—, el breaker transiciona a **Half-Open** (semi-abierto): permite pasar un número limitado de llamadas de prueba para verificar si el proveedor se recuperó. Si las llamadas de prueba son exitosas, el breaker cierra. Si fallan, vuelve a abrir.

La librería `pybreaker` implementa este patrón en Python con una interfaz limpia. `CircuitBreaker(fail_max=5, reset_timeout=30)` crea un breaker que abre después de cinco fallos y entra en Half-Open después de treinta segundos. La anotación `@breaker` sobre la función que llama al proveedor aplica el breaker automáticamente: `pybreaker` captura las excepciones de la función, cuenta los fallos, y lanza `CircuitBreakerError` cuando el breaker está abierto sin llamar a la función original. El código del Gateway que llama a la función decorada captura `CircuitBreakerError` y ejecuta el fallback apropiado.

El design correcto del sistema de breakers usa breakers separados por proveedor: un breaker para Anthropic, otro para OpenAI, otro para Google. Esto permite que el sistema falle al proveedor secundario cuando el primario tiene el breaker abierto, en lugar de fallar toda la pila de LLM cuando un solo proveedor tiene problemas. La combinación de Circuit Breaker y fallback de proveedor es el mecanismo de alta disponibilidad más efectivo para sistemas de IA: cuando Anthropic está degradado, el sistema detecta automáticamente el problema (breaker abre), y las llamadas que antes irían a Anthropic ahora van a OpenAI (fallback), todo sin intervención manual.

Los listeners del Circuit Breaker son el mecanismo de observabilidad del patrón. Un `CircuitBreakerListener` puede implementar `state_change(cb, old_state, new_state)` para recibir notificaciones cuando el estado cambia: emitir una métrica de Prometheus (`circuit_breaker_state_changes_total`), enviar un evento a PagerDuty cuando el breaker se abre, y registrar la recuperación cuando se cierra. El historial de estado del breaker —cuántas veces abrió, cuánto tiempo estuvo abierto, qué fallos lo dispararon— es información operativa que informa la evolución del sistema: si el breaker de un proveedor específico abre varias veces por semana, el equipo tiene evidencia para investigar si el problema es del proveedor o del sistema.

Configuración e implementación del Circuit Breaker

- **Parámetros del Circuit Breaker:** `fail_max` (número de fallos para abrir, típicamente 5-10), `reset_timeout` (segundos en Open antes de Half-Open, típicamente 30-60 segundos), `exclude` (lista de excepciones que no cuentan como fallos, como `ValidationError` del schema de respuesta que es un error del sistema, no del proveedor).
- **Breakers separados por proveedor:** `anthropic_breaker = CircuitBreaker(fail_max=5, reset_timeout=30)`, `openai_breaker = CircuitBreaker(fail_max=5, reset_timeout=30)`; el Gateway usa el breaker del proveedor activo y hace failover al proveedor secundario si el breaker está abierto.
- **Integración en el Gateway:**

```
try: response = await call_with_breaker(provider, messages) except CircuitBreakerError: if fallback_provider: return await self.complete_with_provider(fallback_provider, messages) else: raise ServiceUnavailableError("All LLM providers unavailable")
```

.
- **Métricas del Circuit Breaker:** `circuit_breaker_state` (gauge, 0=Closed/1=Open/2=Half-Open), `circuit_breaker_state_changes_total` (counter por proveedor y transición), `circuit_breaker_rejected_calls_total` (counter de llamadas

rechazadas por breaker abierto); monitorizar para detectar proveedores con problemas sistémicos.

- **Alertas de operación:** alerta inmediata (PagerDuty/Slack) cuando el breaker de cualquier proveedor abre; alerta de recuperación cuando cierra; alerta de breaker en Half-Open durante más de cinco minutos (indica un proveedor con recuperación intermitente).

Nota del Arquitecto: El timeout de `reset_timeout` del Circuit Breaker es un parámetro de ingeniería que debe calibrarse con datos de los incidentes reales del proveedor. Si los incidentes de Anthropic típicamente duran entre dos y diez minutos, un `reset_timeout` de treinta segundos va a producir múltiples ciclos de Open→Half-Open→Open durante el incidente, generando ruido de alertas. Un `reset_timeout` de dos a tres minutos reduce el ruido y da tiempo al proveedor de recuperarse antes de probar. Revisar los postmortems del proveedor disponibles en su página de status para calibrar este parámetro con datos reales.

El Circuit Breaker no es solo un mecanismo de resiliencia técnica: es un sistema de información sobre la confiabilidad de los proveedores externos del sistema. El historial de aperturas del breaker es evidencia objetiva de la calidad de servicio real del proveedor —más objetiva que los SLAs publicitados—, y esa evidencia debe informar las decisiones de arquitectura sobre diversificación de proveedores y estrategias de fallback.

Principio de resiliencia: El Circuit Breaker protege al sistema de los efectos cascada de un proveedor degradado: sin él, un proveedor lento genera timeouts que consumen threads y aumentan la latencia de todo el sistema; con él, el sistema falla rápido con un mensaje de error apropiado y continúa operando via el proveedor de fallback.

Módulo 5 – Capítulo 10 – Sección 05

Patrones de composición: encadenamiento secuencial, paralelo y condicional

Los sistemas de IA reales raramente consisten en una sola llamada al LLM. Un sistema de análisis de documentos puede necesitar: extraer las entidades relevantes del documento con una llamada, clasificar el tipo de documento con otra llamada simultánea, y generar un resumen ejecutivo con una tercera llamada que depende de los resultados de las dos anteriores. La composición de múltiples operaciones de LLM en un flujo coherente —con manejo explícito de dependencias, errores y estado parcial— es el dominio de los patrones de composición. La diferencia entre un sistema bien compuesto y uno mal compuesto es la diferencia entre un pipeline donde cada paso conoce exactamente qué recibe y qué produce, y un pipeline donde el estado viaja como dictionaries anidados de significado ambiguo que solo el autor original puede interpretar.

El encadenamiento secuencial es el patrón más común: la salida de cada paso es la entrada del siguiente. La implementación correcta tipifica explícitamente la salida de cada paso con Pydantic: el `Step1Output` se convierte en el `Step2Input` a través de un mapping explícito, no de `response["field"]` sobre un dict anónimo. Esta tipificación hace que el flujo sea comprensible para cualquier miembro del equipo sin necesidad de rastrear el dict a través de múltiples funciones. En LCEL, el operador `|` encadena `Runnable` components de forma que el output del primero alimenta el input del segundo, con type checking en tiempo de ejecución.

El encadenamiento paralelo —fan-out/fan-in— es el patrón para operaciones independientes que pueden ejecutarse concurrentemente. `asyncio.gather(*[step_a(input), step_b(input), step_c(input)])` ejecuta las tres operaciones en paralelo y espera a que todas terminen antes de continuar. La latencia total del paso paralelo es la latencia del paso más lento, no la suma de todas las latencias: si cada llamada tarda 2 segundos, el paso paralelo tarda 2 segundos en lugar de 6. En LCEL, `RunnableParallel(key_a=chain_a, key_b=chain_b)` implementa fan-out/fan-in con manejo de errores integrado. La semántica de `asyncio.gather` con `return_exceptions=True` permite que el paso paralelo continúe

incluso si alguna rama falla, con el error capturado como valor en lugar de excepción propagada.

El encadenamiento condicional —routing— selecciona qué rama del pipeline ejecutar según el resultado de un paso anterior. El paso de clasificación produce una categoría, y el router selecciona el paso de procesamiento correcto para esa categoría: `{"legal": legal_chain, "financiamiento": financiero_chain, "general": general_chain}[document_type]`. El routing puede ser determinístico (basado en el resultado tipado del clasificador) o probabilístico (basado en la distribución de confianza del clasificador). La implementación correcta hace el routing explícito en el código en lugar de delegarlo al LLM vía instrucciones de "si el documento es X, haz Y", porque el routing explícito es testeable, loggeable y modificable sin cambiar el prompt.

Los checkpoints y la reanudación son el mecanismo que hace que los pipelines largos sean tolerantes a fallos. En un pipeline de diez pasos donde cada paso tarda 3 segundos, un fallo en el paso 8 descarta 21 segundos de trabajo completado. Guardar el output de cada paso en Redis — `redis.set(f"pipeline:{request_id}:step_{n}", json.dumps(step_output), ex=3600)` — permite reanudar el pipeline desde el último checkpoint en lugar de reiniciar desde el principio. LangGraph implementa checkpointing como característica nativa de su `StateGraph`, con backends de persistencia en memoria, SQLite, o PostgreSQL.

Patrones de composición en la práctica

- **Tipos explícitos entre pasos:** `class Step1Output(BaseModel): entities: List[str]; class Step2Input(BaseModel): entities: List[str];` transformar explícitamente entre tipos con `step2_input = Step2Input(entities=step1_output.entities)` en lugar de pasar dicts anónimos; el type checking estático detecta incompatibilidades antes de llegar a producción.
- **Fan-out/fan-in tipado:** `results = await asyncio.gather(step_a(input), step_b(input), return_exceptions=True)`; verificar si cada resultado es una excepción con `isinstance(r, Exception)` antes de usarlo; decidir si la falla de una rama es fatal o si el pipeline puede continuar con las ramas exitosas.
- **Router explícito en código:** `HANDLERS: Dict[str, Runnable] = {"legal": legal_chain, "financiamiento": financiero_chain}; handler = HANDLERS.get(classification, default_chain)`; loggear el valor de `classification` y el `handler` seleccionado para cada request como métrica operativa.

- **Checkpointing con Redis:** guardar el output de cada paso con `redis.set(f"pipeline:{request_id}:step_{step_num}", serialized_output, ex=ttl_seconds)` antes de continuar; en la reanudación, verificar qué pasos ya tienen checkpoint y empezar desde el primero que falta; el TTL debe ser mayor que el tiempo máximo total del pipeline.
- **LangGraph para pipelines con ciclos:** cuando el flujo requiere ciclos (reintento de un paso hasta que cumple una condición, evaluación de calidad que puede redirigir a un paso anterior), LangGraph es la herramienta correcta; un `StateGraph` con `add_conditional_edges` implementa el routing condicional con ciclos de forma explícita y testeable.

Nota del Arquitecto: El antipatrón más común en composición de LLM es el "god prompt": un prompt único que intenta hacer todo lo que el pipeline haría en múltiples pasos. El god prompt es atractivo por su aparente simplicidad pero produce sistemas frágiles: un cambio en un aspecto del prompt puede romper otro aspecto no relacionado, el debugging es difícil porque no hay pasos intermedios observables, y el testing es prácticamente imposible porque el output es holístico y no verificable paso a paso. La descomposición en pasos explícitos con tipos entre ellos es siempre superior a largo plazo.

Un pipeline de composición bien diseñado es un sistema donde cada paso tiene una única responsabilidad, una interfaz tipada de entrada y salida, y un comportamiento de error definido explícitamente. Esta claridad de diseño es lo que permite al equipo evolucionar el pipeline —reemplazar un paso, agregar una rama, cambiar el modelo de un paso específico— sin entender el comportamiento completo del sistema cada vez que se hace un cambio.

Principio de composición: La composición de múltiples llamadas al LLM en pipelines es más mantenible cuando cada paso tiene una única responsabilidad, tipos explícitos de entrada y salida, y un comportamiento de error definido; la alternativa del "god prompt" que intenta hacer todo en una llamada produce sistemas que no pueden testearse, observarse ni evolucionarse de forma independiente en cada parte.

Módulo 5 – Capítulo 10 – Sección 06

Cierre: diseñando aplicaciones de IA mantenibles y evolucionables

El módulo 5 comenzó con una pregunta implícita: ¿cómo se convierte un modelo de lenguaje —un sistema de predicción probabilística de texto— en una aplicación de software que otros equipos pueden desplegar, mantener, y evolucionar de forma independiente? La respuesta que este módulo ha construido capítulo por capítulo no es una colección de técnicas aisladas: es una arquitectura donde cada capa tiene una responsabilidad clara, una interfaz definida, y mecanismos de observabilidad que la hacen comprensible para el equipo que la opera.

Los patrones de este último capítulo —validación de entrada, parseo estructurado de salida, Gateway centralizado, Circuit Breaker, composición tipada— son los que convierten el código de integración con LLMs de prototipos que funcionan en notebooks a servicios que pueden desplegarse en producción sin insomnio. Un sistema construido con estos patrones soporta el cambio de proveedor sin modificar el código de la aplicación (Gateway), falla rápido y de forma informativa cuando el proveedor tiene problemas (Circuit Breaker), produce datos tipados que el resto de la aplicación puede consumir de forma confiable (parseo estructurado), y rechaza inputs malformados o maliciosos antes de gastar tokens en procesarlos (validación de entrada).

La propiedad que define si una aplicación de IA es mantenible no es la cantidad de features que tiene, sino la velocidad a la que el equipo puede cambiar cualquier componente sin romper el resto. Esta velocidad depende directamente de cuán bien el sistema implementa los principios que este módulo ha desarrollado: separación de concerns (el Gateway no conoce la lógica de negocio; la lógica de negocio no conoce los detalles del proveedor), contratos explícitos entre componentes (los tipos Pydantic entre pasos del pipeline), observabilidad en cada capa (logging estructurado, trazas distribuidas, métricas de costo y calidad), y un sistema de evaluación que detecta regresiones antes de que lleguen a producción (el pipeline de CI/CD con quality gates del Capítulo 6).

La propiedad que define si una aplicación de IA es evolucionable es diferente: es la capacidad de incorporar nuevas capacidades del ecosistema —nuevos modelos, nuevas APIs, nuevos frameworks— sin reestructurar el sistema existente. El patrón Gateway facilita la incorporación de nuevos proveedores. El patrón adapter —encapsular cada LLM detrás de una interfaz común— facilita el cambio de modelo. Los patrones de composición tipada facilitan la extensión del pipeline con nuevos pasos sin afectar los existentes. El strangler fig pattern del Capítulo 4 facilita la migración incremental de features existentes hacia nuevas implementaciones.

Síntesis de los principios del Módulo 5

- **Ingeniería en la capa de infraestructura:** retry con backoff, circuit breaker, logging estructurado, métricas de costo y calidad son responsabilidades de la infraestructura del sistema, no del código de negocio; el Gateway centraliza estos concerns para que el código de negocio permanezca limpio.
- **Contratos tipados en cada interfaz:** Pydantic entre la aplicación y el LLM (parseo de salida), Pydantic entre pasos del pipeline (composición tipada), configuración YAML versionada (modelos y prompts como código); los contratos explícitos son la base de la mantenibilidad a largo plazo.
- **Observabilidad desde el sprint 0:** trazas con OpenTelemetry, logging con structlog, métricas de costo por request; la instrumentación retroactiva cuesta diez veces más en tiempo y produce observabilidad incompleta.
- **Evaluación como disciplina de ingeniería:** el dataset de evaluación curado, las métricas de RAGAS, y el pipeline de CI con quality gates son la única forma de saber si un cambio de prompt, modelo o configuración mejora o degrada el sistema.
- **Diseño para el cambio:** los proveedores de LLM cambian sus modelos, precios y APIs con regularidad; un sistema bien diseñado puede absorber estos cambios en el Gateway y en la capa de configuración sin afectar la lógica de negocio.

Hacia el Módulo 6: Sistemas RAG

Los conceptos del Módulo 5 son los prerequisites directos de los Sistemas RAG que constituyen el Módulo 6. Esta continuidad no es accidental: RAG (Retrieval-Augmented Generation) es la aplicación más importante del ecosistema de LLMs para aplicaciones empresariales, y requiere todos los fundamentos que el Módulo 5 ha construido.

El **flujo de indexación de LlamaIndex** desarrollado en el Capítulo 3 — `IngestionPipeline` con `SentenceSplitter`, `VectorStoreIndex`, `SimpleDirectoryReader` — es la base sobre la que el Módulo 6 construye pipelines RAG completos. La diferencia es que en el Módulo 6 el pipeline de indexación tiene consideraciones adicionales: estrategias de chunking semántico vs estructural, enriquecimiento de chunks con metadata para filtrado, indexación incremental de documentos nuevos sin re-indexar la colección completa.

Las **métricas de RAGAS** —`faithfulness`, `answer_relevancy`, `context_recall`— introducidas en los Capítulos 5 y 7 son el sistema de evaluación específico de RAG. En el Módulo 6 se profundiza en su interpretación: `faithfulness` mide si la respuesta está soportada por el contexto recuperado (fundamental para detectar alucinaciones en RAG), `context_recall` mide si los chunks recuperados contienen la información necesaria para responder (el indicador de calidad del retriever), `answer_relevancy` mide si la respuesta responde la pregunta del usuario (el indicador de calidad del generador). La interpretación conjunta de estas tres métricas permite diagnosticar si el problema de un sistema RAG está en el retriever, en el re-ranker, o en el generador.

La **integración con pgvector y Qdrant** presentada en el Capítulo 4 es la capa de infraestructura sobre la que los sistemas RAG operan en producción. En el Módulo 6 se añaden las consideraciones de escala: índices IVFFlat vs HNSW en pgvector para diferentes tradeoffs de velocidad y recall, la configuración de `ef_construction` y `m` en HNSW para optimizar el recall del vector search, y las estrategias de sharding para colecciones con más de diez millones de vectores. Qdrant como alternativa especializada ofrece filtrado por metadata más eficiente que pgvector para colecciones heterogéneas donde el filtrado por tenant, fecha o categoría es parte de cada query.

El **pipeline de evaluación automatizada** del Capítulo 6 —GitHub Actions con quality gates sobre métricas de RAGAS— es el mecanismo de CI/CD específico para sistemas RAG. En el Módulo 6 este pipeline se extiende con la evaluación de los componentes individuales del pipeline RAG: el retriever evaluado por `context_recall` sobre un conjunto de preguntas con contextos de referencia conocidos, el re-ranker evaluado por la mejora en el ranking de los chunks relevantes, y el generador evaluado por `faithfulness` sobre los chunks recuperados por el retriever real (no el ideal). Esta evaluación por componente permite identificar qué parte del pipeline RAG es el cuello de botella de calidad y dirigir los esfuerzos de optimización de forma precisa.

El Módulo 6 asume que el lector conoce cómo hablar con APIs de LLM con los patrones de resiliencia correctos (Módulo 5, Capítulos 1-4), cómo evaluar la calidad de las respuestas con

métricas automatizadas y LLM-as-judge (Módulo 5, Capítulos 5 y 7), y cómo desplegar cambios de forma progresiva con quality gates automáticos (Módulo 5, Capítulo 6). Estos fundamentos son los que permiten al Módulo 6 moverse directamente a los desafíos específicos de RAG: el diseño del pipeline de retrieval, la evaluación del re-ranker, y la optimización de los parámetros de chunking y búsqueda vectorial con evidencia empírica.

"Any fool can write code that a computer can understand. Good programmers write code that humans can understand." — Martin Fowler. En AI Engineering, la máxima tiene una extensión: los buenos ingenieros de IA escriben sistemas que los humanos pueden entender, los equipos pueden operar, las herramientas de evaluación pueden medir, y el negocio puede confiar. La diferencia entre un prototipo impresionante y un sistema de producción confiable no es la complejidad del modelo ni el tamaño del contexto: es la ingeniería que rodea al modelo.

Síntesis final del Módulo 5: Las aplicaciones de IA mantenibles y evolucionables no se construyen encontrando el prompt perfecto o el modelo más capaz disponible; se construyen aplicando los mismos principios de ingeniería de software que producen cualquier sistema de calidad: separación de responsabilidades, contratos explícitos entre componentes, observabilidad en cada capa, y evaluación continua de la calidad.